PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DUOC UC

ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE (ERS)

Sistema de Gestión y Venta de Gas

"Distribuidora de Gas El Volcán"

Asignatura	DSY1104 — Desarrollo Fullstack II
Integrantes	Nicolás Arancibia Pablo Gómez Benjamín González
Profesor(a)	Hernán Saavedra
Versión	2.1

CHILLÁN · REGIÓN DE ÑUBLE · CHILE

Especificación de Requerimientos de Software (ERS)

Sistema de Gestión y Venta de Gas — “Distribuidora de Gas El Volcán”

Asignatura: DSY1104 — Desarrollo Fullstack II **Evaluación:** Evaluación Parcial N° 2 (migración del sitio a React) **Tecnología principal de esta entrega:** React (SPA) **Versión del documento:** 2.1 (actualización de la EP2: interfaz y catálogo unificado) **Fecha:** Octubre 2026

Control de versiones

Versión	Fecha	Descripción	Autor
1.0	EP1	ERS inicial del sitio estático (HTML/CSS/JS)	Equipo
2.0	**EP2**	**Actualización: la solución se reespecifica como SPA en React, se agregan requisitos de testing y arquitectura de componentes**	**Equipo**
2.1	**EP2**	**Actualización de la interfaz (identidad visual, diseño responsive y CSS modular por componente) y del catálogo público unificado: los productos creados por el administrador ahora se reflejan en el sitio del cliente.**	**Equipo**

---

1. Introducción

1.1 Propósito

Este documento especifica los **requerimientos funcionales y no funcionales** del sistema de gestión y venta de gas de la empresa *Distribuidora de Gas El Volcán*. La versión 2.0 **actualiza el ERS de la Evaluación Parcial N° 1**, donde el sitio era estático (HTML/CSS/JS), para reflejar la nueva arquitectura basada en **React (SPA)**, la separación de responsabilidades en componentes, la persistencia temporal en el navegador y el plan de integración con backend de microservicios.

El ERS sirve como:

- Base para el diseño y la implementación del frontend.
- Referencia para las pruebas unitarias y el aseguramiento de calidad.
- Acuerdo entre los interesados (clientes, operadora, repartidores, administración) y el equipo de desarrollo.

1.2 Alcance de esta entrega

Dentro del alcance (EP2):

- Migración completa del sitio a **React** con componentes, props y `state`.
- Diseño **responsive** con Bootstrap (móvil, tableta y escritorio) e **identidad visual** de marca (portada con imagen de cilindros, tarjetas de beneficios con iconos y navegación con pestaña activa)

- **CSS modular** por componente (`navbar.css` , `productos.css` , `carrito.css`) sobre una hoja base (`estilos.css`).
- **Catálogo público unificado**: catálogo base + ediciones + **productos creados** en el panel de administración.
- Flujo de compra con **carrito** persistente y **checkout**.
- Módulo de **administración** (dashboard, inventario, usuarios).
- **Pruebas unitarias automatizadas** con Vitest + React Testing Library y reporte de cobertura.
- Persistencia temporal en el **navegador** (`localStorage`), en ausencia de backend.

Fuera del alcance de esta entrega (planificado para etapas posteriores):

- Backend con **microservicios Spring Boot** y APIs REST/JSON.
- Base de datos relacional (MySQL/PostgreSQL/Oracle) normalizada (3FN).
- Autenticación segura con **tokens** y control de acceso real en el servidor (**RBAC**).
- **Seguimiento de pedidos en tiempo real** con mapas (Leaflet/Google Maps).
- Despliegue en **AWS** con **Docker**.

1.3 Definiciones y acrónimos

Término	Significado
SPA	*Single Page Application*; aplicación de una sola página que se actualiza dinámicamente.
Componente	Unidad reutilizable de interfaz en React que recibe `props` y puede manejar `state` .
Props	Propiedades de solo lectura que un componente recibe de su componente padre.
State	Estado interno y mutable de un componente, gestionado con `useState` .
RBAC	*Role-Based Access Control*; control de acceso basado en roles.
CRUD	Crear, Leer, Actualizar y Eliminar.
RF / RNF	Requerimiento Funcional / No Funcional.
API REST	Interfaz HTTP que expone recursos y responde en JSON.
Carrito	Conjunto de productos seleccionados por un cliente antes de confirmar el pedido.

1.4 Referencias

- Documento de contexto del proyecto *DSY1104 — Forma C — Distribuidora de Gas El Volcán*.
- Rúbrica de la *Evaluación Parcial N° 2* (DSY1104).
- Documento de cobertura de testing del proyecto (`DOCUMENTO_COBERTURA_TESTING.md`).

---

2. Descripción general

2.1 Contexto del negocio

Distribuidora de Gas El Volcán es una empresa familiar ubicada en **Chillán, Región de Ñuble**, fundada en 1998. Distribuye cilindros de gas licuado (5, 11 y 15 kg, además de accesorios) a clientes **residenciales y**

comerciales. Cuenta con 2 camiones, 3 repartidores, 1 operadora de llamadas y 1 administradora, y recibe entre **80 y 120 pedidos diarios**.

El proceso actual es **manual** (teléfono y cuadernos), lo que genera: clientes sin información del estado del pedido, pedidos duplicados o perdidos, falta de historial, descontrol del stock y diferencias de caja. El sistema busca **digitalizar** el catálogo, los pedidos y la administración.

2.2 Objetivos del sistema

1. Permitir que los clientes **vean el catálogo** y **realicen pedidos** de forma autónoma.
2. Dar a la administración una vista **centralizada** de productos, stock y usuarios.
3. Registrar de forma **automática** los pedidos (órdenes) y las solicitudes de contacto.
4. Sentar las bases para, en etapas posteriores, **seguir el pedido en tiempo real** y aplicar **RBAC**.
5. Ofrecer una experiencia **responsive**, simple y consistente.

2.3 Actores y roles (RBAC)

Rol	Descripción	Permisos previstos
Cliente	Compra cilindros y accesorios.	Ver catálogo, armar carrito, registrarse, iniciar sesión y confirmar pedidos.
Administrador	Gestiona el sistema.	Todo: inventario (CRUD de productos), usuarios (ver/crear/filtrar), reportes y órdenes.
Operadora / Despachadora	Recibe y asigna pedidos *(rol planificado)*.	Ver todos los pedidos del día, asignarlos a repartidores y cambiar su estado.
Repartidor	Realiza entregas *(rol planificado)*.	Ver solo sus pedidos asignados y actualizar su estado.

> **Estado actual (EP2):** el sistema diferencia **Cliente** y **Administrador** en el frontend (el login redirige al panel admin cuando corresponde). La **protección efectiva de rutas** y la verificación de token corresponden a la etapa con backend. Los roles *Operadora* y *Repartidor* quedan **especificados** pero aún no implementados.

2.4 Supuestos y dependencias

- La conectividad de los repartidores es **intermitente**; por ello el diseño prioriza simplicidad y tolerancia a fallos.
- Los repartidores usan **teléfonos Android de gama media-baja**; la interfaz debe ser liviana.
- En esta entrega **no existe backend**, por lo que la persistencia se realiza en `localStorage`.
- La autenticación actual es **de demostración** (no cifra contraseñas); su endurecimiento depende del backend.

---

3. Arquitectura de la solución

3.1 Stack tecnológico obligatorio

Capa	Tecnología	Rol
Frontend	**React (SPA)**	Interfaz completa.
Diseño	**Bootstrap 5 + CSS modular**	Responsive (≥360, ≥768, ≥1280 px); hoja base (`estilos.css`) y hojas por componente (`navbar.css` , `productos.css` , `carrito.css`).
Iconografía	**Bootstrap Icons**	Iconos de beneficios (portada) y de acciones de la interfaz.
Ruteo	**React Router**	Navegación entre vistas sin recargar.
Estado	**Hooks (`useState` , `useEffect`)**	Carrito, formularios y filtros.
Persistencia (EP2)	`localStorage`	Simula la base de datos mientras no hay API.
Backend *(planificado)*	**Spring Boot (Java)**	Microservicios con API REST/JSON.
Base de datos *(planificado)*	**MySQL**	Modelo relacional (3FN).
Mapas *(planificado)*	**Leaflet / Google Maps**	Seguimiento del pedido.
Cloud *(planificado)*	**AWS + Docker**	Despliegue.
Pruebas	**Vitest** + React Testing Library	Pruebas unitarias y cobertura (`npm test` / `test:coverage`).

3.2 Arquitectura del frontend (componentes)

El frontend se organiza en **páginas** (vistas asociadas a una ruta) y **componentes reutilizables**, apoyándose en **utilidades puras** para la lógica de negocio.

```
main.jsx
└── App (estado global del carrito + enrutador)
    ├── LayoutPublico  → BotonTienda + Navbar + <Outlet/> + Footer
    │   ├── Inicio
    │   ├── Productos        → Producto (componente)
    │   ├── DetalleProducto
    │   ├── Carrito
    │   ├── Ventas (checkout)
    │   ├── Nosotros
    │   ├── Consejos
    │   ├── Tiendas
    │   ├── Contacto
    │   ├── Login
    │   └── Registro
    └── AdminLayout (panel, menú lateral) → <Outlet/>
        ├── Dashboard
        ├── Inventario
        ├── ListaUsuarios
        └── NuevoUsuario
```

Utilidades (lógica pura y reutilizable):

- `utils/validaciones.js`: validación de RUN, teléfono, correo, formularios y helpers de `localStorage`.
- `utils/carrito.js`: operaciones puras del carrito (`agregarItem`, `decrementarItem`, `calcularTotal`, etc.).
- `utils/productos.js`: catálogo de productos. `catalogoActual()` (base + ediciones del administrador) y `productosConCreados()` (catálogo actual + **productos creados**), que es la fuente **unificada** del sitio público y del panel.

Estilos e identidad visual: Bootstrap 5 se complementa con **CSS modular por componente** y **Bootstrap Icons** (ver §3.6).

3.3 Ruteo y layout

- **Parte pública** (`LayoutPublico`): incluye `Navbar` (barra superior *sticky* con **pestaña activa** resaltada mediante `NavLink` y contador del carrito) y `Footer` en todas las vistas.
- **Panel de administración** (`AdminLayout`): layout propio con menú lateral colapsable (botón *hamburguesa* en móvil) y cierre de sesión.
  - Vistas del panel: **Dashboard** (indicadores), **Ordenes**, **Clientes** y **Reportes** (maquetas con datos de ejemplo), **Inventario** (CRUD) y **Usuarios**. Las rutas `/admin/ordenes`, `/admin/clientes` y `/admin/reportes` se agregaron en esta integración final.
- El carrito y el total de ítems viven en `App` y se comunican por **props** a `Navbar` y a las páginas.

3.4 Gestión del estado

- **Estado global (de aplicación):** el **carrito**, en `App`, persistido en `localStorage` y reflejado en la `Navbar`.

- **Estado local:** cada formulario (Login, Registro, Contacto, Inventario, NuevoUsuario) y los filtros (categorías de productos, filtro de usuarios) usan `useState`.
- **Flujo de datos:** *props* hacia abajo (`Producto` recibe `onAgregar`, `Navbar` recibe `totalItems`) y *callbacks* hacia arriba para modificar el carrito.

3.5 Visión full-stack (planificada)

En etapas posteriores, el frontend consumirá vía HTTP (GET/POST/PUT/DELETE) **al menos dos microservicios Spring Boot**:

- **Servicio de Catálogo/Inventario:** productos, stock y precios.
- **Servicio de Pedidos/Usuarios:** registro, autenticación, órdenes y estados.

No se permite comunicación directa del frontend con la base de datos. Cada servicio tendrá su propia configuración, su base de datos y responderá en **JSON**.

3.6 Identidad visual y diseño de la interfaz

La interfaz mantiene una identidad visual coherente con la marca **El Volcán**:

- **Navegación:** barra superior *sticky* con estilo propio (`navbar-gas`), que resalta la **sección activa** mediante `NavLink`.
- **Portada (Inicio):** mensaje de bienvenida, imagen de **cilindros de gas** y una tarjeta con **4 beneficios** ilustrados con **Bootstrap Icons** —*entrega a domicilio*, *cilindros certificados*, *equipo local* y *cobertura en Chillán*— en una grilla responsive (`xs=12`, `sm=6`, `lg=3`).
- **Catálogo y carrito:** tarjetas de producto y controles con estilo propio (`productos.css`, `carrito.css`), botones unificados (`btn-login`) y esquinas redondeadas.
- **CSS modular:** los estilos de cada sección viven en su propia hoja (`navbar.css`, `productos.css`, `carrito.css`), lo que reduce el CSS global y facilita el mantenimiento.

3.7 Catálogo unificado (público ↔ administración)

El catálogo que ve el cliente es el **mismo** que administra el panel. La función `productosConCreados()` combina:

1. el **catálogo base** (productos precargados),
2. las **ediciones** hechas por el administrador (`edicionesProductos`), y
3. los **productos creados** en el panel (`productos`).

Esta fuente única alimenta el **catálogo público**, el **detalle**, el **carrito**, el **checkout** y los **indicadores del dashboard**. Así, cualquier producto creado o editado por el administrador se refleja de inmediato en el sitio del cliente (ver **RF-37**).

---

4. Requisitos funcionales (RF)

Prioridad: **A = Alta**, **M = Media**, **B = Baja**. Estado: ✅ **Implementado (EP2)**, 🟡 **Parcial**, ⬜ **Planificado**.

4.1 Módulo público y catálogo

Cód.	Requerimiento	Prioridad	Estado
RF-01	El sistema debe mostrar una página de **inicio** con un mensaje de bienvenida, la imagen de cilindros, los **beneficios del servicio** (iconos) y acceso directo al catálogo.	A	✅
RF-02	El sistema debe mostrar el **catálogo público unificado** (base, ediciones y **productos creados** por el administrador) en tarjetas, con imagen, nombre, descripción, precio y stock.	A	✅
RF-03	El usuario debe poder **filtrar** los productos por **categoría** (Cilindros de Gas, Reguladores, Mangueras y Conexiones, Accesorios) y ver "Todos".	A	✅
RF-04	El usuario debe poder abrir el **detalle** de un producto con toda su información (código, categoría, precios residencial/comercial y stock).	A	✅
RF-05	El sistema debe indicar claramente cuando un producto **no existe**.	M	✅
RF-06	El sistema debe mostrar precios formateados en **pesos chilenos** (separador de miles).	M	✅
RF-37	Todo **producto creado** por el administrador debe reflejarse de inmediato en el **catálogo público**, el **detalle**, el **carrito** y el **checkout**, y contabilizarse en los indicadores del panel.	A	✅

4.2 Módulo carrito

Cód.	Requerimiento	Prioridad	Estado
RF-07	El usuario debe poder **agregar** un producto al carrito desde el catálogo y desde el detalle.	A	✅
RF-08	El usuario debe poder **aumentar** y **disminuir** la cantidad de cada ítem del carrito.	A	✅
RF-09	Al disminuir a **cero**, el ítem debe **eliminarse** del carrito.	M	✅
RF-10	El sistema debe **calcular el total** aplicando tarifa **residencial** o **comercial** según el tipo de cliente.	A	✅
RF-11	La barra de navegación debe mostrar el **contador de ítems** del carrito.	M	✅
RF-12	El carrito debe **persistir** entre recargas de la página.	M	✅
RF-13	El sistema debe indicar cuando el carrito está **vacío**.	B	✅

4.3 Módulo de cuenta (registro / sesión)

Cód.	Requerimiento	Prioridad	Estado
RF-14	El usuario debe poder **registrarse** como cliente (Residencial o Comercial) con validación de campos.	A	✅
RF-15	El sistema debe **validar** RUN (7–9 dígitos), teléfono, formato y dominio de correo, contraseña (4–10) y términos.	A	✅
RF-16	El sistema debe **impedir correos duplicados** en el registro.	A	✅
RF-17	El usuario debe poder **iniciar sesión**; si es administrador, se redirige al panel admin.	A	✅
RF-18	El sistema debe **cerrar sesión** eliminando los datos de sesión.	A	✅
RF-19	El sistema debe usar **selectores dependientes** Región → Comuna.	M	✅

4.4 Módulo de pedido / checkout

Cód.	Requerimiento	Prioridad	Estado
RF-20	Para confirmar un pedido, el usuario debe tener **sesión iniciada**; si no, se lo invita a iniciar sesión o registrarse.	A	✅
RF-21	El usuario debe poder **revisar** su pedido (ítems, cantidades y subtotales) antes de confirmar.	A	✅
RF-22	El usuario debe ingresar **dirección de despacho** y **comuna**, y elegir **método de pago** (Efectivo / Tarjeta).	A	✅
RF-23	El sistema debe **generar una orden** con número, fecha, cliente, dirección, ítems y total, y **registrarla**.	A	✅
RF-24	Tras confirmar, el sistema debe **simular la pasarela de pago**: en éxito mostrar una **confirmación** con el número de orden, el total y **vaciar el carrito**; en rechazo mostrar la vista **"No se pudo realizar el pago"** sin guardar la orden.	A	✅
RF-25	El sistema debe mostrar el estado del pedido al cliente *(tiempo real con mapas)*.	M	⬜

4.5 Módulo de contacto e información

Cód.	Requerimiento	Prioridad	Estado
RF-26	El usuario debe poder enviar un **mensaje de contacto** (nombre, correo, teléfono, asunto, mensaje) con validación.	M	✅
RF-27	El sistema debe **registrar las solicitudes** de contacto.	M	✅

Cód.	Requerimiento	Prioridad	Estado
RF-28	El sistema debe ofrecer páginas informativas: **Nosotros**, **Consejos** y **Tiendas** (con mapa referencial).	B	✅

4.6 Módulo de administración

Cód.	Requerimiento	Prioridad	Estado
RF-29	El panel de administración debe tener un **dashboard** con indicadores (ventas del día, en ruta, clientes nuevos, stock crítico) y últimas órdenes.	A	✅
RF-30	El administrador debe poder **listar el inventario** con código, nombre, categoría, stock y precios, resaltando el **stock crítico**.	A	✅
RF-31	El administrador debe poder **crear** productos con validación (código único, nombre, precios, stock entero, categoría); el producto creado queda **disponible en el catálogo público** (ver RF-37).	A	✅
RF-32	El administrador debe poder **editar** productos (tanto base como creados).	A	✅
RF-33	El administrador debe poder **eliminar** los productos creados.	M	✅
RF-34	El administrador debe poder **listar y filtrar usuarios** por rol (Administradores, Clientes, Vendedores).	A	✅
RF-35	El administrador debe poder **crear usuarios** con rol, región/comuna y validaciones.	A	✅
RF-36	La edición de productos base debe **conservar** el catálogo original (base + ediciones) sin sobrescribirlo.	M	✅

5. Requisitos no funcionales (RNF)

Cód.	Categoría	Requerimiento
RNF-01	**Responsive**	La interfaz debe verse correctamente en móvil (≥360 px), tableta (≥768 px) y escritorio (≥1280 px); los menús se colapsan en "hamburguesa" y las grillas se reorganizan según el tamaño (los beneficios de la portada pasan de 4 a 2 y a 1 columna).
RNF-02	**Usabilidad**	Los formularios deben ser simples y con **pocos pasos**; los errores se muestran en español de forma clara.
RNF-03	**Compatibilidad**	Debe funcionar en navegadores modernos (Chrome, Edge, Firefox) y en teléfonos Android de gama media-baja.
RNF-04	**Rendimiento**	La aplicación debe cargar rápido incluso con conexión lenta o inestable; el bundle de producción debe estar optimizado.
RNF-05	**Seguridad**	Ninguna vista/endpoint debe quedar accesible sin autenticación (salvo login y registro). Las contraseñas deben tratarse de forma segura en el backend.

Cód.	Categoría	Requerimiento
RNF-06	**Confiabilidad**	La lectura de datos persistidos debe ser **tolerante a errores** (JSON corrupto no debe romper la app).
RNF-07	**Mantenibilidad**	Código organizado en componentes y utilidades reutilizables; estilo consistente y **lint** sin errores.
RNF-08	**Testabilidad**	La lógica de negocio debe poder probarse de forma aislada; el proyecto debe incluir **pruebas unitarias** y **cobertura**.
RNF-09	**Accesibilidad**	Uso de roles, etiquetas y `aria-label` básicos (por ejemplo, en botones y notificaciones).
RNF-10	**Escalabilidad**	La arquitectura debe permitir conectar el frontend a **microservicios** sin reescribir la interfaz.
RNF-11	**Disponibilidad**	El despliegue final debe realizarse en **AWS** con **Docker**.
RNF-12	**Privacidad**	Los datos de clientes y pedidos son confidenciales; solo el personal autorizado debe acceder a ellos.

6. Modelo de datos (preliminar)

Entidades principales y sus atributos clave:

Entidad	Atributos principales
Usuario	run, nombre, correo, teléfono, región, comuna, contraseña (hash), rol, estado, fecha de registro.
Producto	id, código, nombre, descripción, precio residencial, precio comercial, stock, stock crítico, categoría, imagen.
Ítem de carrito	id de producto, cantidad.
Orden / Pedido	número, fecha, cliente, correo, dirección, comuna, método de pago, ítems, total, estado.
Solicitud de contacto	fecha, nombre, correo, teléfono, asunto, mensaje.

> En una etapa posterior, el modelo relacional se normalizará a **3FN** y se distribuirá entre los microservicios.

7. Interfaces externas

7.1 APIs REST previstas (planificadas)

Método	Ruta	Descripción
`GET`	`/api/productos`	Lista el catálogo.
`POST`	`/api/productos`	Crea un producto (admin).
`PUT`	`/api/productos/{id}`	Actualiza un producto (admin).
`DELETE`	`/api/productos/{id}`	Elimina un producto (admin).
`POST`	`/api/auth/login`	Inicia sesión y devuelve token.
`POST`	`/api/usuarios`	Registra/crea usuario.
`GET`	`/api/usuarios`	Lista usuarios (admin).
`POST`	`/api/pedidos`	Crea un pedido.
`GET`	`/api/pedidos/{id}`	Consulta el estado de un pedido.
`PUT`	`/api/pedidos/{id}/estado`	Actualiza el estado del pedido.

7.2 Servicios externos

- **Mapas:** Leaflet / Google Maps para mostrar la ubicación de sucursales (implementado como mapa embebido referencial) y, en etapas posteriores, el seguimiento de la entrega.

---

8. Restricciones

Tipo	Descripción
Conectividad	Zonas con señal intermitente: la interfaz debe tolerar conexiones lentas.
Dispositivos	Teléfonos Android de gama media-baja; sin computadores para los repartidores.
Nivel técnico	Usuarios sin experiencia en software de gestión: mínima cantidad de pasos.
Motor de BD	Sin preferencia: MySQL, PostgreSQL u Oracle.
Privacidad	Datos de clientes y pedidos confidenciales.
Stack	React (SPA), Spring Boot, API REST/JSON, MySQL, AWS + Docker.

---

9. Matriz de trazabilidad (RF ↔ implementación ↔ pruebas)

RF	Implementación (frontend EP2)	Prueba asociada
RF-01, RNF-01	`pages/Inicio.jsx`, `components/Navbar.jsx` (`navbar.css`), CSS modular (`productos.css`, `carrito.css`)	`Navbar.test.jsx` (props/render)
RF-02, RF-06	`pages/Productos.jsx`, `components/Producto.jsx`	`Producto.test.jsx` (props/DOM + mock)
RF-03	`pages/Productos.jsx` (filtro por categoría)	— *(integración)*
RF-04, RF-05	`pages/DetalleProducto.jsx`	— *(integración)*
RF-07 – RF-10, RF-12	`utils/carrito.js`, `pages/Carrito.jsx`, `App.jsx`	`carrito.test.js` (lógica pura)
RF-11	`components/Navbar.jsx`	`Navbar.test.jsx` (prop `totalItems`)
RF-14 – RF-16, RF-19	`pages/Registro.jsx`, `utils/validaciones.js`	`validaciones.test.js`
RF-17, RF-18	`pages/Login.jsx`, `AdminLayout.jsx`	`Login.test.jsx` (formulario interactivo)
RF-20 – RF-24	`pages/Ventas.jsx`	— *(integración)*
RF-26, RF-27	`pages/Contacto.jsx`, `utils/validaciones.js`	`validaciones.test.js`
RF-28	`pages/Nosotros.jsx`, `Consejos.jsx`, `Tiendas.jsx`	`Nosotros.test.jsx` (estado)
RF-29, RF-30	`pages/admin/Dashboard.jsx`, `Inventario.jsx` (indicadores y stock crítico)	— *(integración)*
Vista admin	`pages/admin/Ordenes.jsx`, `Clientes.jsx`, `Reportes.jsx` (paneles con datos de ejemplo, integrados por el compañero)	— *(integración, maquetas)*
RF-31 – RF-33, RF-36	`pages/admin/Inventario.jsx`, `utils/productos.js`	`productos.test.js` (CRUD)
RF-34, RF-35	`pages/admin/ListaUsuarios.jsx`, `NuevoUsuario.jsx`	— *(integración)*
RF-37	`pages/Productos.jsx`, `Carrito.jsx`, `DetalleProducto.jsx`, `Ventas.jsx`, `admin/Dashboard.jsx`, `utils/productos.js`	`productos.test.js` (catálogo unificado)

10. Criterios de aceptación (resumen)

1. Todas las vistas cargan **sin errores** en consola y se adaptan a los tres tamaños de pantalla.
2. Es posible **agregar productos**, modificar cantidades y ver el **total correcto** en el checkout.

3. El **registro** y el **login** funcionan con sus validaciones; el administrador accede al panel.
4. El administrador puede **crear/editar/eliminar** productos y **crear/filtrar** usuarios; los productos **creados** aparecen en el catálogo público (RF-37).
5. La lógica de negocio y los componentes clave están cubiertos por **pruebas unitarias** que pasan y generan **reporte de cobertura**.
6. `npm run build` y `npm run lint` finalizan **sin errores**.
7. La interfaz presenta una identidad visual coherente (portada, navegación con pestaña activa, iconos) y se adapta a móvil, tableta y escritorio (RF-01, RNF-01).

---

11. Glosario

- **Carrito:** lista temporal de productos que un cliente desea comprar.
- **Orden / Pedido:** registro confirmado de una compra, con número, ítems y total.
- **Stock crítico:** nivel de existencias que activa una alerta de reposición.
- **Tarifa residencial / comercial:** precio aplicado según el tipo de cliente.
- **Catálogo unificado:** catálogo base + ediciones + productos creados, expuesto por igual al sitio público y al panel de administración.
- **RBAC:** control de acceso basado en roles.
- **Cobertura:** porcentaje del código ejecutado por las pruebas.

Fin del documento.